



网络通信

余 燁

本章学习提示

- 本章学习Java语言在网络通讯方面的应用
  - 网络基本概念
  - java.net中类的使用
  - 简单的socket程序设计

目　录

1 网络通信

2 URL编程

3 Socket网络编程

计算机网络

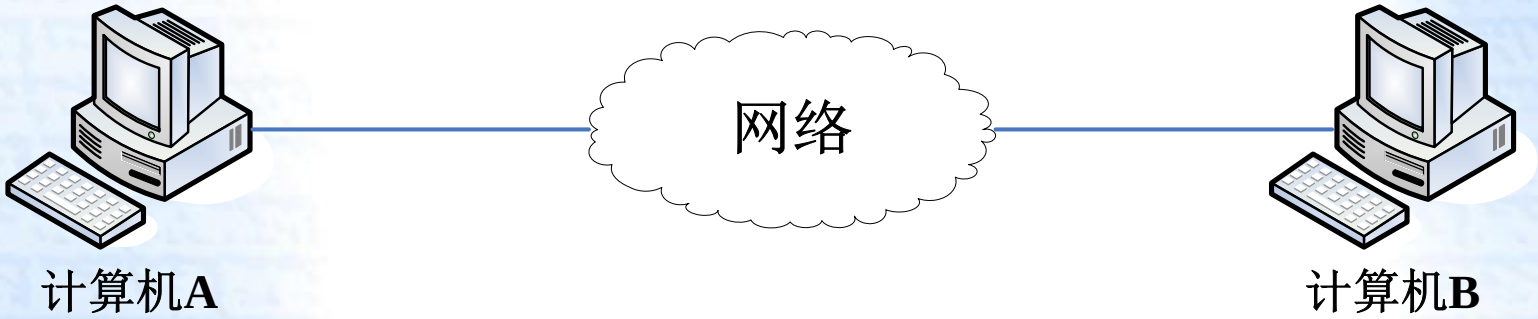
- **网络**就是把分布在不同地理区域的计算机与专门的外部设备用通信线路互连成一个规模大、功能强的系统，从而使众多的计算机可以方便地互相传递信息，共享课件、软件、数据信息等资源
- 网络提供以下主要功能：
  - 资源共享
  - 信息传输和集中处理
  - 均衡负载与分布处理
  - 综合信息服务

计算机网络

- 局域网（LAN，Local Area Network）
  - 指在一个小地理范围内的各种计算机网络设备互连在一起的通信网络，可以包含一个或多个子网，通常局限在几千米的范围之内。
- 城域网（MAN，Metropolitan Area Network）
  - 主要由城域范围内的各局域网之间互连而构成的，现在很少提起这个概念
- 广域网（WAN，Wide Area Network）
  - 是由相距较远的局域网或城域网互连而成，通常是除了计算机设备以外，还要涉及一些电信通讯方式。

计算机网络

- 需要解决的问题



- 和谁进行通信？——IP地址、端口
- 如何通信？——遵循一定的规则：协议

IP地址

- 网络中每台主机都有一个IP地址（逻辑地址）
- IP地址可以通过ipconfig/all 来查看

```
无线局域网适配器 WLAN:

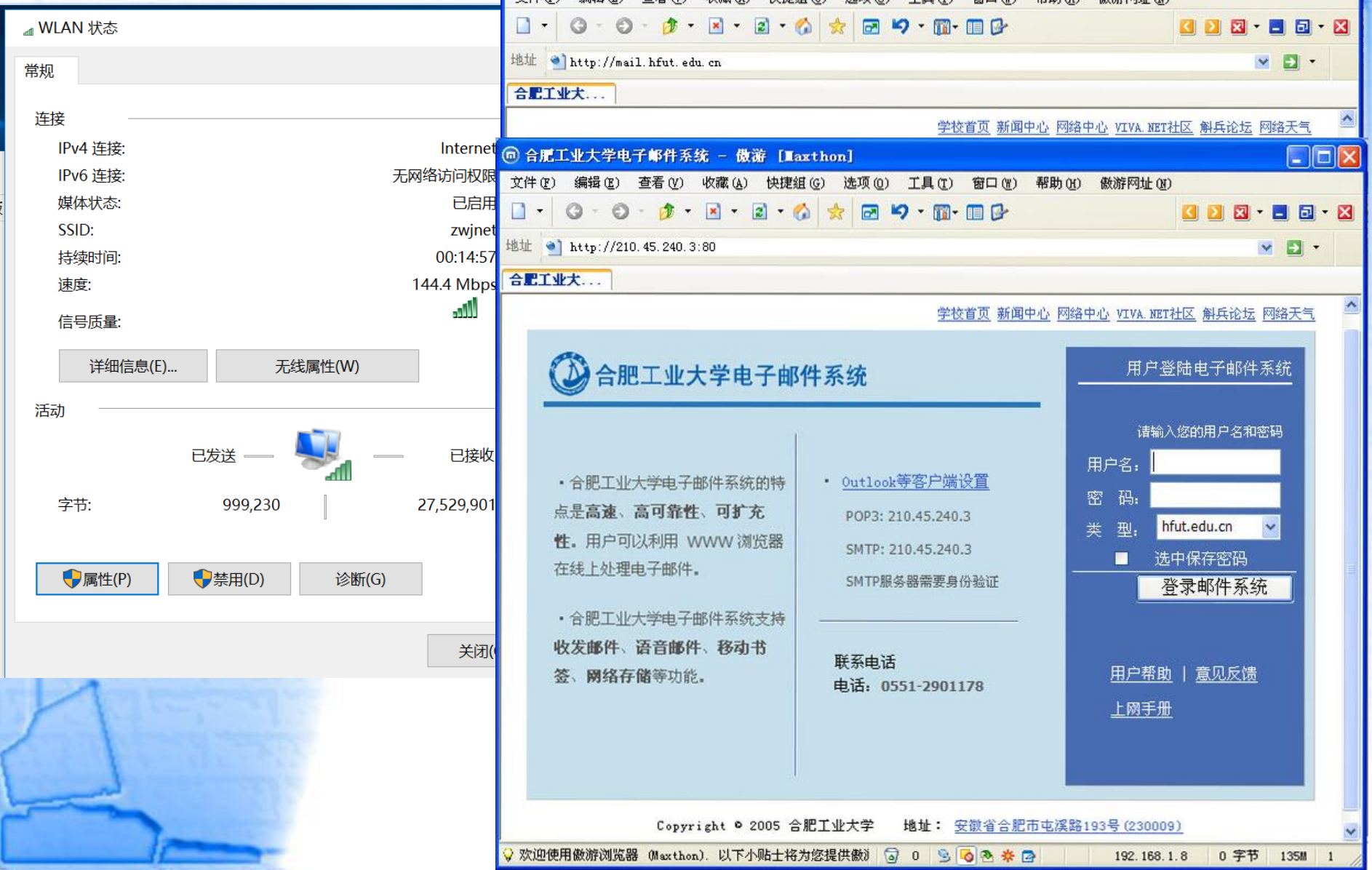
   连接特定的 DNS 后缀 . . . . . . . :
   描述. . . . . . . . . . . . . . . : Intel(R) Dual Band Wireless-AC 8265
   物理地址. . . . . . . . . . . . . : F8-94-C2-E5-6F-14
   DHCP 已启用 . . . . . . . . . . . : 是
   自动配置已启用. . . . . . . . . . : 是
   本地链接 IPv6 地址. . . . . . . . : fe80::70b1:fabf:cc79:4a85%16(首选)
   IPv4 地址 . . . . . . . . . . . . : 192.168.31.230(首选)
   子网掩码 . . . . . . . . . . . . : 255.255.255.0
   获得租约的时间 . . . . . . . . . : 2018年10月7日 14:10:05
   租约过期的时间 . . . . . . . . . : 2018年10月8日 3:15:17
   默认网关. . . . . . . . . . . . . : 192.168.31.1
   DHCP 服务器 . . . . . . . . . . . : 192.168.31.1
   DHCPv6 IAID . . . . . . . . . . . : 133731522
   DHCPv6 客户端 DUID . . . . . . . . : 00-01-00-01-22-B1-47-BE-54-E1-AD-DE-D8-FB
   DNS 服务器 . . . . . . . . . . . : 192.168.31.1
   TCPIP 上的 NetBIOS . . . . . . . . : 已启用
```

- ping 127.0.0.1 （ip地址）可以测试计算机是否连接到某目标ip上

IP地址

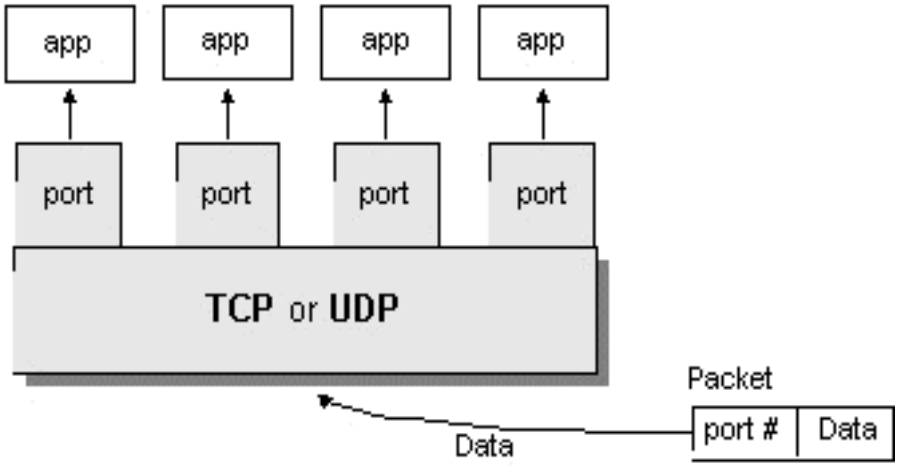
- IPv4是一个32位的二进制数。由于不便于记忆将此地址表示成4个十进制数，各取0-255的值，每个值之间用点“.”号分隔，如：192.168.1.2（**点分十进制的格式**）
- 最新的IPv6则使用128位的数字来表示地址，可以容纳更多的网络设备，如： ABCD:EF01:2345:6789:ABCD:EF01:2345:6789（**冒分十六进制表示法**）
- 为了进一步方便使用，人们给每个节点都起一个名字，把名字与IP地址建立一个对应关系，这就是域名系统**DNS**(Domain Name System)，凡域名空间有定义的域名都可以有效地转换成IP地址

IP地址 和 域名之例


WLAN 状态
常规
连接
IPv4 连接:
IPv6 连接:
媒体状态:
SSID:
持续时间:
速度:
信号质量:
详细信息(E)...
无线属性(W)
活动
已发送
已接收
字节:
999,230
27,529,901
属性(P)
禁用(D)
诊断(G)
关闭(
合肥工业大学电子邮件系统 - 傲游 [Maxthon]
文件(F) 编辑(E) 查看(V) 收藏(A) 快捷组(G) 选项(O) 工具(T) 窗口(W) 帮助(H) 傲游网址(M)
地址 http://mail.hfut.edu.cn
合肥工业大...
学校首页 新闻中心 网络中心 VIVA.NET社区 斛兵论坛 网络天气
合肥工业大学电子邮件系统 - 傲游 [Maxthon]
文件(F) 编辑(E) 查看(V) 收藏(A) 快捷组(G) 选项(O) 工具(T) 窗口(W) 帮助(H) 傲游网址(M)
地址 http://210.45.240.3:80
合肥工业大...
学校首页 新闻中心 网络中心 VIVA.NET社区 斛兵论坛 网络天气
合肥工业大学电子邮件系统
• 合肥工业大学电子邮件系统的特点是高速、高可靠性、可扩充性。用户可以利用 WWW 浏览器在线上处理电子邮件。
• 合肥工业大学电子邮件系统支持收发邮件、语音邮件、移动书签、网络存储等功能。
• Outlook等客户端设置
POP3: 210.45.240.3
SMTP: 210.45.240.3
SMTP服务器需要身份验证
联系电话
电话：0551-2901178
用户登陆电子邮件系统
请输入您的用户名和密码
用户名:
密 码:
类 型:
hfut.edu.cn
选中保存密码
登录邮件系统
用户帮助 | 意见反馈
上网手册
Copyright © 2005 合肥工业大学 地址： 安徽省合肥市屯溪路193号 (230009)
欢迎使用傲游浏览器 (Maxthon). 以下小贴士将为您提供傲游
192.168.1.8
0 字节
135M

端口(PORT)

- 任何一个应用程序发布之后，如果需要被用户所访问，那么必定要指定端口的信息（端口号用来指明该计算机上的应用程序）。
- 端口号用2个字节表示，范围为0-65535, 其中低于1024的端口号保留给那些已定义的服务, 不要使用。
- 在同一个协议下，端口号不能重复；不同协议下，可以重复。



端口(PORT)

- 公认端口：0-1023之间，在一些操作系统中必须管理员才能申请这些端口
  - 21：FTP协议，用于文件传输
  - 22：SSH协议，用于安全的远程连接
  - 25：SMTP，简单邮件传输协议，用于发送电子邮件
  - 80：HTTP，超文本传输协议
- 注册端口：1024-49151，松散绑定一些服务
  - Tomcat服务器，端口号是8080
  - Oracle服务器，端口号是1521
  - Mysql数据库，端口号是3306
- 动态端口/私有端口：49152-65535，应用程序使用的动态端口，一般应用程序不主动使用它

统一资源定位符

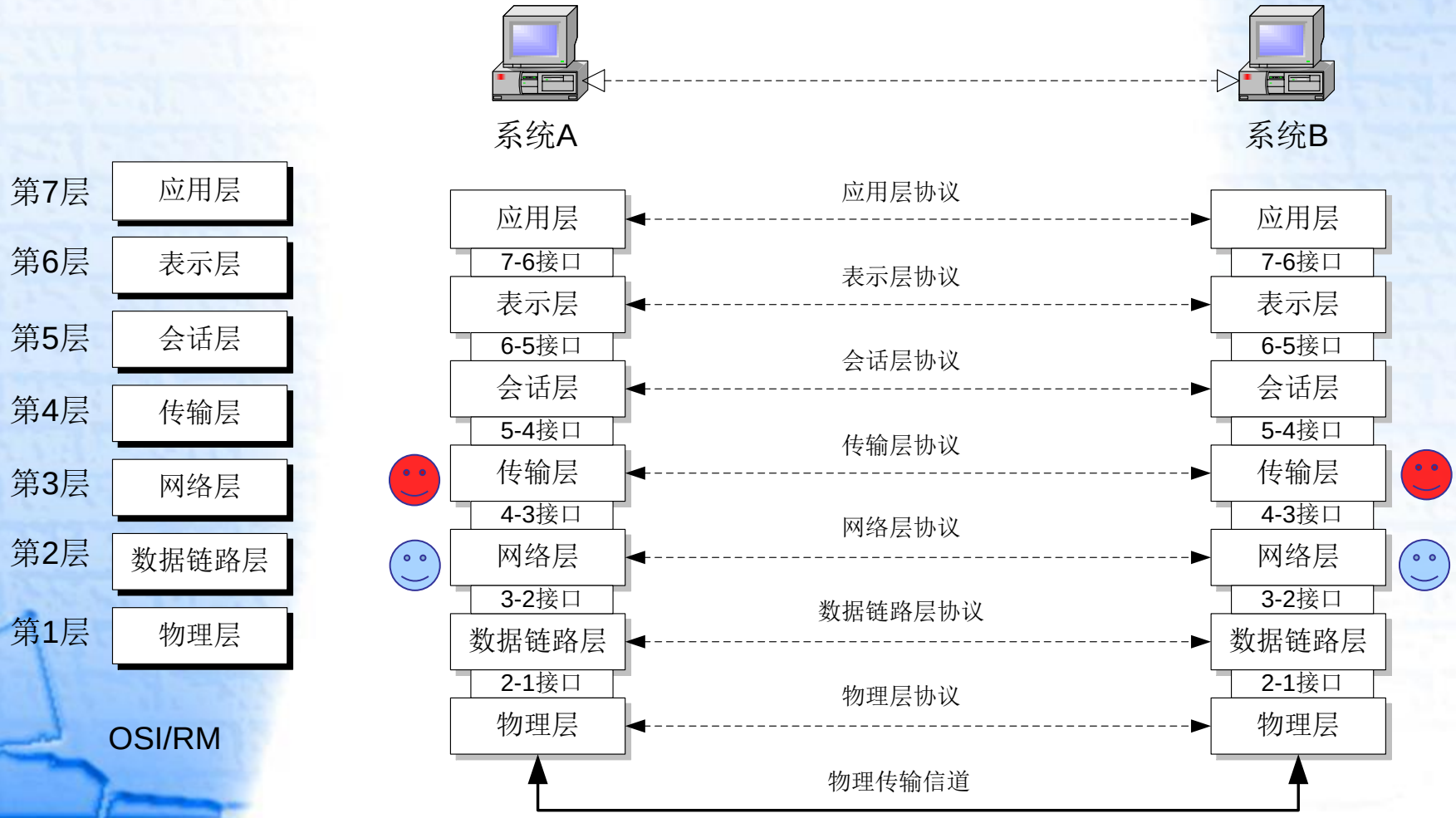
- 在WWW上，每一信息资源都有统一且唯一的地址，该地址就叫URL (Uniform Resource Locator)，是WWW的统一资源定位标识符。
- URL由4 部分组成：**资源类型（协议）**、存放资源的**主机（域名或ip地址）**、**端口**和**资源文件名**。

  例如，http://email.hfut.edu.cn:80/index.html
  - http ： 超文本 传输协议，表示该资源类型是超文本信息
  - email.hfut.edu.cn：

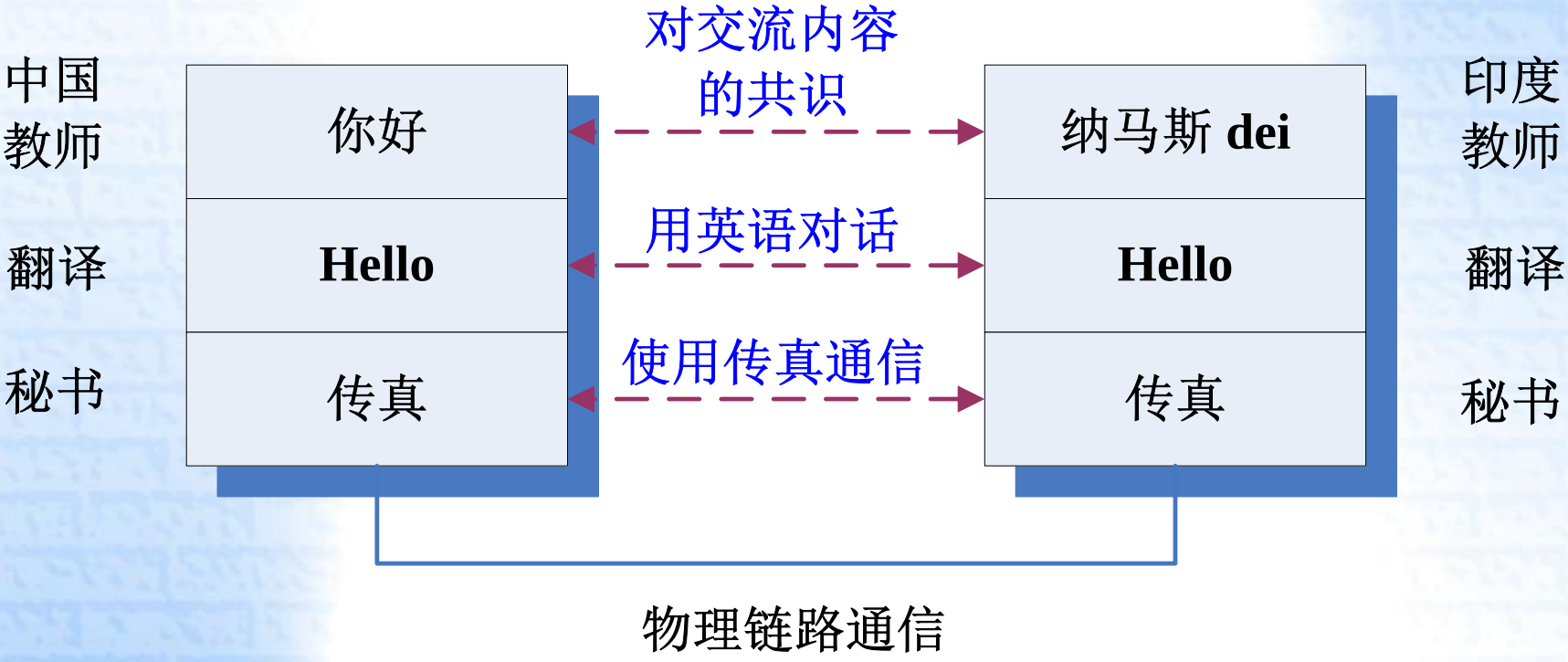
    是合肥工业大学邮件服务器的主机域名
  - 80：表示HTTP协议的端口号（默认是80，通常会被省略）
  - index.html：表示资源文件名

计算机网络的分层模型

- **OSI** (Open System Interconnection)理论推荐分层模型，又称为开放系统互连模型



对等通信示例


中国
教师
翻译
秘书
你好
Hello
传真
对交流内容
的共识
用英语对话
使用传真通信
纳马斯 dei
Hello
传真
印度
教师
翻译
秘书
物理链路通信

OSI分层模型各层含义

- 应用层
  - 最顶层，主要是为了实现不同业务而开发的各种协议，比如HTTP、FTP、SSH等，程序员往往比较关心这个层面的协议
- 表示层
  - 主要用于处理两个通信系统中交换信息的表示方式，比如加密、压缩、格式转换等
- 会话层
  - 在两个节点之间建立端连接
- 传输层
  - 为会话层用户提供一个端到端的可靠、透明和优化的数据传输服务机制

OSI分层模型各层含义

- 网络层
  - 建立节点的联系，路由选择、建立和维护连接、拥塞控制、计算，通常是交换机
- 数据链路层
  - 将数据分帧、处理流控制、差错控制、指定拓扑结构、硬件寻址、网卡、网桥、交换机
- 物理层
  - 传输电子信号，集线器、中继器、调制解调器、网线、双绞线、同轴电缆

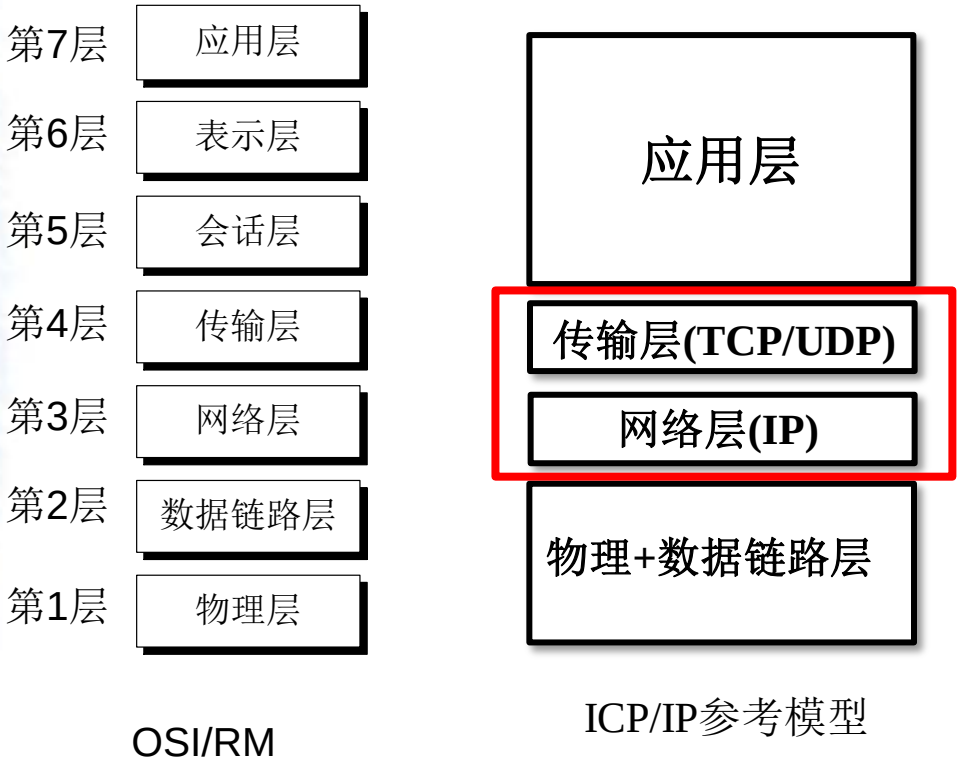
网络分层的原因

- 由于结点之间联系很复杂，在制定协议时，把复杂成分分解为一些简单的成分，再将他们复合起来。
- 最常用的复合方式是层次方式，即同层间可以通信，上一层可以调用下一层，而与再下一层不发生关系。

只关心实现部分，不关心具体的细节。

TCP/IP模型

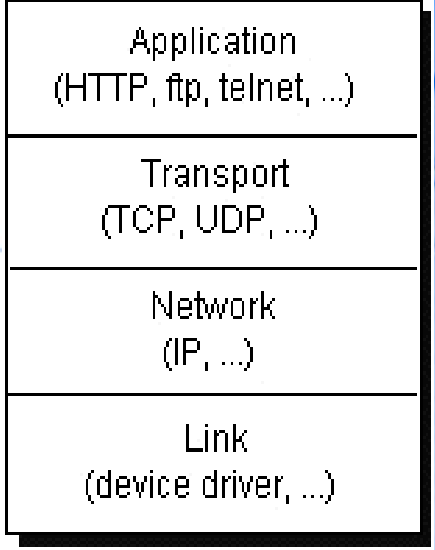
- 为了进行简化，TCP/IP把前面三层合并为应用层
- 物理+数据链路层，由操作系统统一进行管理



各层所使用的协议

- 应用层：远程登录协议Telnet、文件传输协议FTP、超文本传输协议HTTP、域名服务DNS、简单邮件传输协议SMTP、邮局协议POP3等
- 传输层：传输控制协议TCP、用户数据报协议UDP
- 网络层：网际协议IP、Internet互联网控制报文协议ICMP、Internet组管理协议IGMP

协 议



- 互联网上的计算机之间的通讯是通过 **TCP** (Transport Control Protocol) 或 **UDP** (User Datagram Protocol)协议
- TCP协议
  - TCP是Internet的基础和核心，一种面向连接的保证可靠传输的协议，得到的是一个顺序无差错的数据流
  - 优点：在通信前要建立连接，所以安全、可靠
  - 缺点：要等待对方回应，速度较慢，效率相对低下
- UDP协议
  - UDP是从一台计算机向另一台计算机发送称为数据报的独立数据包的协议，该协议并不保证数据报是否能正确地到达目的地。它是一个非面向连接的协议
  - UDP不需要建立连接，所以速度较快，效率高
  - 不能保证传送的数据包按顺序到达，因此安全性较差
  - QQ等IM软件，一般均采用UDP协议

Java与网络1——相关的类

- Java语言取得成功的领域之一就是网络

  (其他语言) 数页代码-->(Java)一条语句

- 在Java中,有一个用来存储internet地址的类叫**InetAddress**.
  - 无构造方法
  - public **static** **InetAddress** **getLocalHost**()

    返回本地主机的 IP 地址
  - public **static** **InetAddress** **getByName**(String host)

    在给定主机名的情况下确定主机的 IP 地址
  - public String **getHostName**()

    获得IP地址所对应的主机名

public final class **Inet4Address** extends InetAddress
public final class **Inet6Address** extends InetAddress

内容详见软件包 java.net

InetAddress 类 示例

```
try
{
   //根据域名得到InetAddress对象
   InetAddress addr =
   InetAddress.getByName("www.hfut.edu.cn");
   System.out.println(addr);
   System.out.println(addr.getHostAddress());
   System.out.println(addr.getHostName());
}
catch (UnknownHostException e)
{  e.printStackTrace();}
```

```
www.hfut.edu.cn/210.45.240.198
210.45.240.198
www.hfut.edu.cn
```

InetAddress 类 示例

```
//使用getLocalHost方法创建InetAddress对象
InetAddress local = InetAddress.getLocalHost();
System.out.println(local);
System.out.println(local.getHostAddress());
System.out.println(local.getHostName());
```

```
Laptop-YU/192.168.6.198
192.168.6.198
Laptop-YU
```

```
//根据IP得到InetAddress对象
InetAddress addr = InetAddress.getByName("210.45.240.71");
System.out.println(addr);
System.out.println(addr.getHostAddress());
System.out.println(addr.getHostName());
```

```
/210.45.240.71
210.45.240.71
www1.hfut.edu.cn
```

TestInetAddress.java

InetSocketAddress类

- 封装端口：在InetAddress的基础上封装端口
- 构造方法
  - InetSocketAddress(InetAddress addr, int port)，根据IP地址和端口号创建套接字地址
  - InetSocketAddress(int port)，创建套接字地址，其中IP地址为通配符地址，端口号为指定值
  - **InetSocketAddress(String hostname, int port)**，根据主机名和端口号创建套接字地址 （用得最多）
- 常用方法
  - InetAddress **getAddress**()，获取InetAddress
  - String **getHostName**()，获取主机名
  - int **getPort**()，获取端口号

InetSocketAddress示例

```
InetSocketAddress address = new InetSocketAddress("localhost", 9999);
System.out.println(address);
System.out.println(address.getHostName());
System.out.println(address.getPort());
InetAddress addr = address.getAddress();
System.out.println (addr);
System.out.println(addr.getHostAddress());
System.out.println(addr.getHostName());
```

```
localhost/127.0.0.1:9999
localhost
9999
localhost/127.0.0.1
127.0.0.1
localhost
```

Java与网络（2）

- Java提供的网络功能有三大类: **URL, Socket, Datagram**.
  - **URL**是三大功能中最高级的一种, 通过URL Java程序可以直接送出或读入网络上的数据.
  - **Socket**是传统网络程序最常用的方式, 可以想象为两个不同的程序通过网络的通信信道通信.
  - **Datagram**是更低级的网络传输方式, 它把数据的目的纪录在数据包中,然后直接放在网络上.

目　录

1 网络通信

2 URL编程

3 Socket网络编程

URL类及其构造方法

- 要使用URL进行通信，就要使用URL类创建其对象，通过引用URL类定义的方法完成网络通信。
- URL类位于java.net包中，其构造方法有：

URL类的构造方法	功能说明
public URL(String str)	使用字符串创建URL对象（绝对路径构建）
public URL(String protocol, String host, String file)	通过指定协议名、主机名、文件名，端口使用默认值，创建URL对象
public URL(String protocol, String host, String port, String file)	通过指定协议名、主机名、文件名和端口号，创建URL对象
public URL(URL content, String str)	通过在已知的URL路径上增加细节的办法创建URL对象（相对路劲构建）

URL的构造示例

```
//利用绝对路径构建URL
URL url = new
URL("http://www.baidu.com:80/index.html#aa");


//利用相对路径构建URL
URL url = new URL("http://www.baidu.com:80/a/");
url = new URL(url, "b.txt");
System.out.println(url.toString());
```

http://www.baidu.com:80/a/b.txt

URL 类的方法

URL 类	功能说明
int getPort()	获得**端口号**,如果端口没有设置，返回-1
String getProtocol()	获得**协议**，如果协议没有设置，返回null
String getHost()	获得**主机名**，如果主机没有设置，返回null
String getFile()	获得**资源名**，如果资源没有设置，返回null
Boolean equals(Object obj)	与指定的URL对象obj 进行比较，如果相同返回true,否则返回false
final InputStream **OpenStream()**	打开到此 URL 的连接并返回一个用于从该连接读入的 InputStream
String toString()	将此URL对象转换为字符串的形式

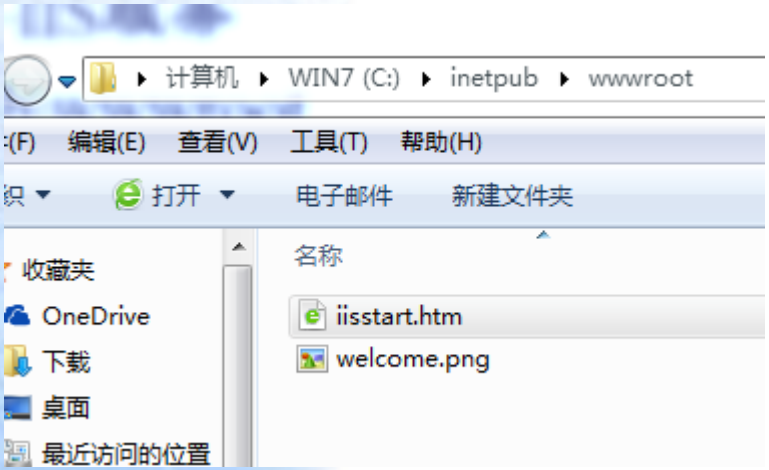
```
URL url = new
URL("http://www.baidu.com:80/index.html#aa?uname=hfut");
System.out.println("协议： " + url.getProtocol());
System.out.println("域名： " + url.getHost());
System.out.println("端口： " + url.getPort());
System.out.println("资源： " + url.getFile());
System.out.println("相对路径： " + url.getPath());
System.out.println("锚点： " + url.getRef());
System.out.println("参数： " + url.getQuery());
```

```
url = new
URL("http://www.baidu.com:80/index.
html?uname=hfut");
System.out.println("参数： " +
url.getQuery());
```

协议：http
域名：www.baidu.com
端口：80
资源：/index.html
相对路径：/index.html
锚点：aa?uname=hfut
参数：null
参数：uname=hfut

开启本地服务器（1）

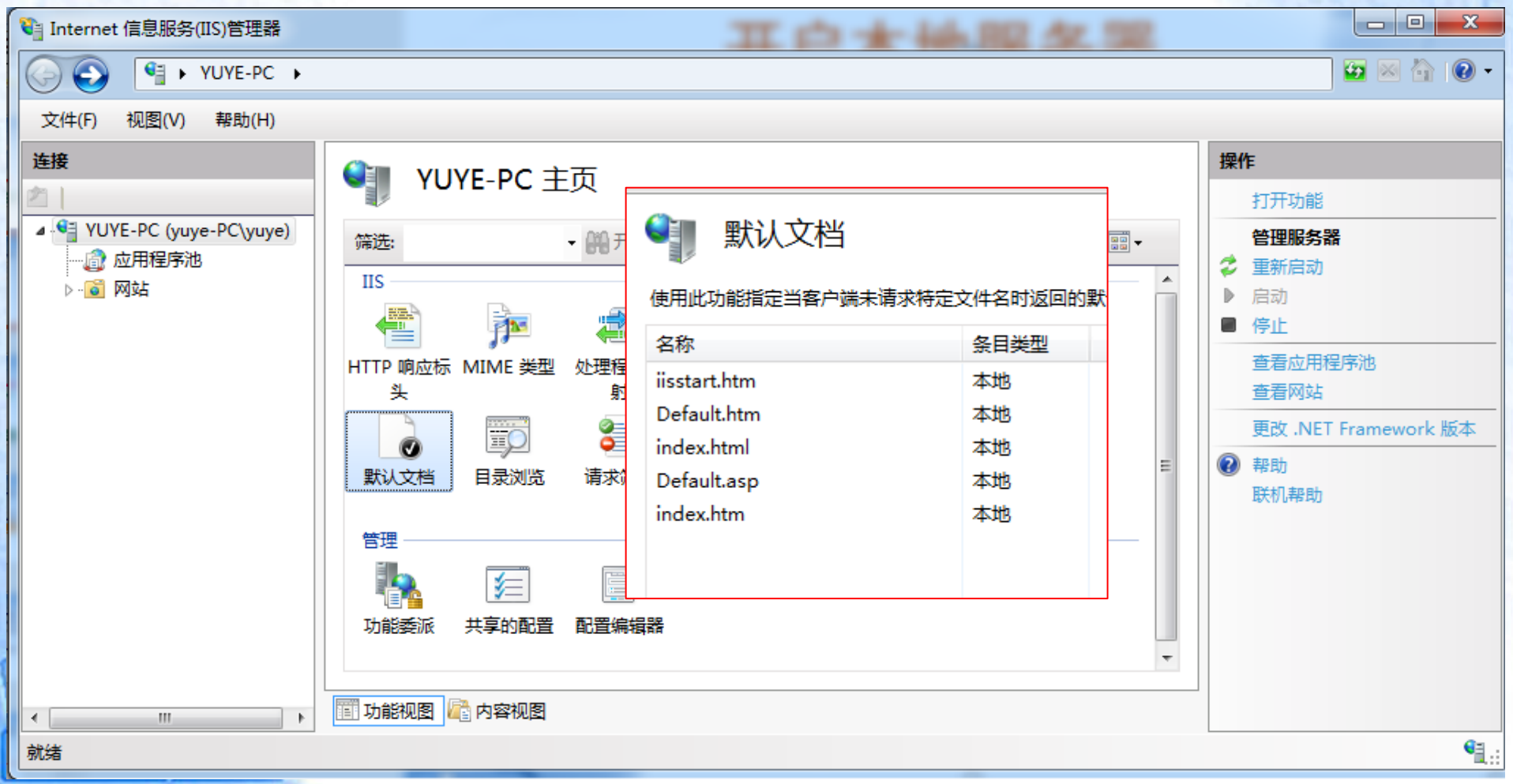
- 控制面板——程序——打开或关闭Windows功能——IIS服务
- C:\inetpub\wwwroot





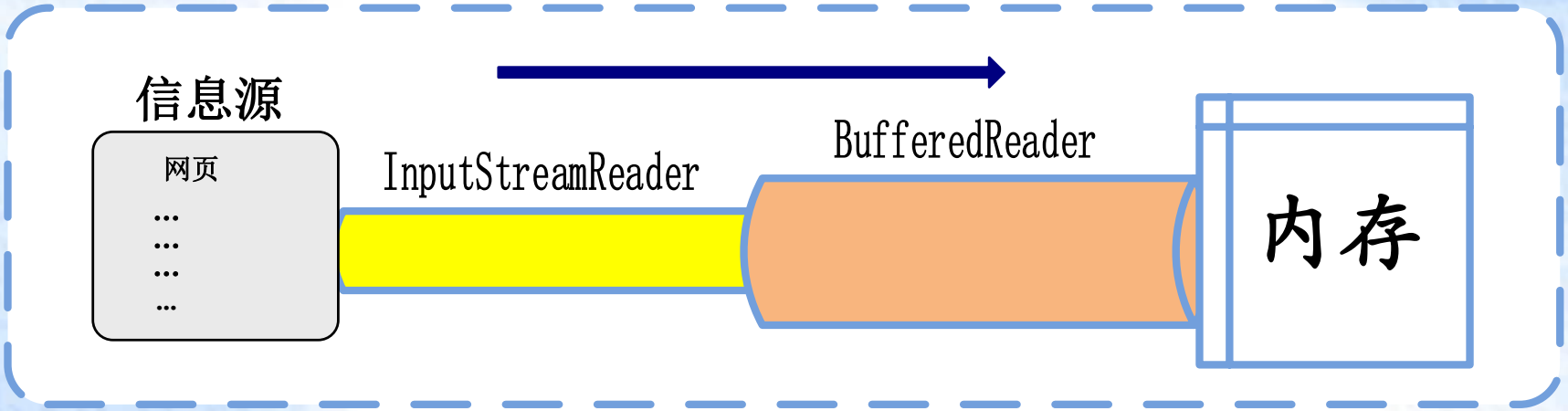
开启本地服务器（2）

- “控制面板”→“系统和安全”→“管理工具”→“Internet 信息服务(IIS)管理器”，打开



URL程序例1

- 例1：获取网页内容并按行打印



代 码

```
URL hfut = new URL("http://127.0.0.1/");
BufferedReader in = new BufferedReader(
        new InputStreamReader( InputStream类的对象) , “utf-8”);
```

hfut.openStream()

```
String inputLine;
//打印输出HTML
while ((inputLine = in.readLine()) != null)
{
    System.out.println(inputLine);
}
//关闭缓冲区
in.close();
```

Network_1.java

URLConnection类

- 使用URL类可以很简单地获得信息，但如果在获取到信息的同时还能向远程的计算机节点**传送信息**，就需要使用URLConnection类。
- 创建URLConnection类的对象，先创建一个URL对象，然后调用该对象的openConnection()方法就会返回一个对应URL地址的URLConnection对象。

内容详见
java.net.URLConnection类

  - //创建URL对象

```
URL url=new URL("http://www.hfut.edu.cn");
```

  - //创建链接通道

```
URLConnection connect=url.openConnection();
```

URLConnection类

- 建立输入/输出流 URLConnection类不仅可以使用getInputStream()方法获得URL节点的信息，还可以采用getOutputStream()方法向URL节点处传送信息。

- 在建立URLConnection类对象的同时就已经在本级和URL节点之上建立了一条HTTP通道。

URL程序例二

- 例：使用java.net.URLConnection连接到互连网，获取网页并按行打印

```
URL hfut = new URL("http:www.hfut.edu.cn");
URLConnection yc =  hfut.openConnection();
BufferedReader in = new BufferedReader(
                    new InputStreamReader( yc.getInputStream() ));

String inputLine;
while ((inputLine = in.readLine()) != null)
System.out.println(inputLine);
in.close();
```

Network_3.java

目　录

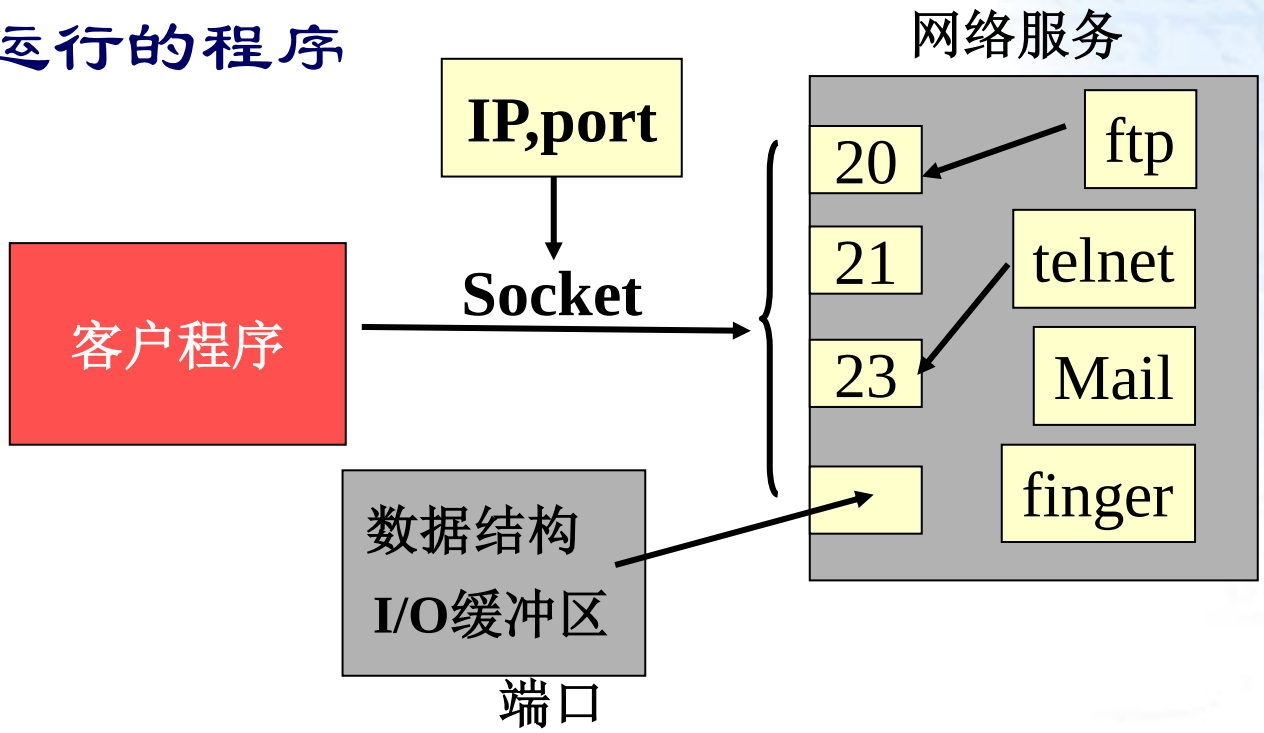
1 网络通信

2 URL编程

3 Socket网络编程

Socket网络编程

- 什么是socket?
  - Socket是网络上运行的程序之间双向通信链路的最后终结点
  - IP与端口的组合得出一个套接字，可以完全分辨internet上运行的程序



ServerSocket类

- 建立服务器
  - 构造方法
    - ServerSocket (int port)
    - ServerSocket(int port, int queueLength)
    - ServerSocket(int port, int queueLength, InetAddress address)
    - 参数：服务器使用的端口号、允许同时连入服务器客户机的数量、服务器的ip地址
  - 常用的方法
    - accept()，接收来自客户机的连接
    - getLocalPort()，获得服务器的端口号
    - getInetAddress()，获得服务器的IP地址
    - close()，关闭Socket

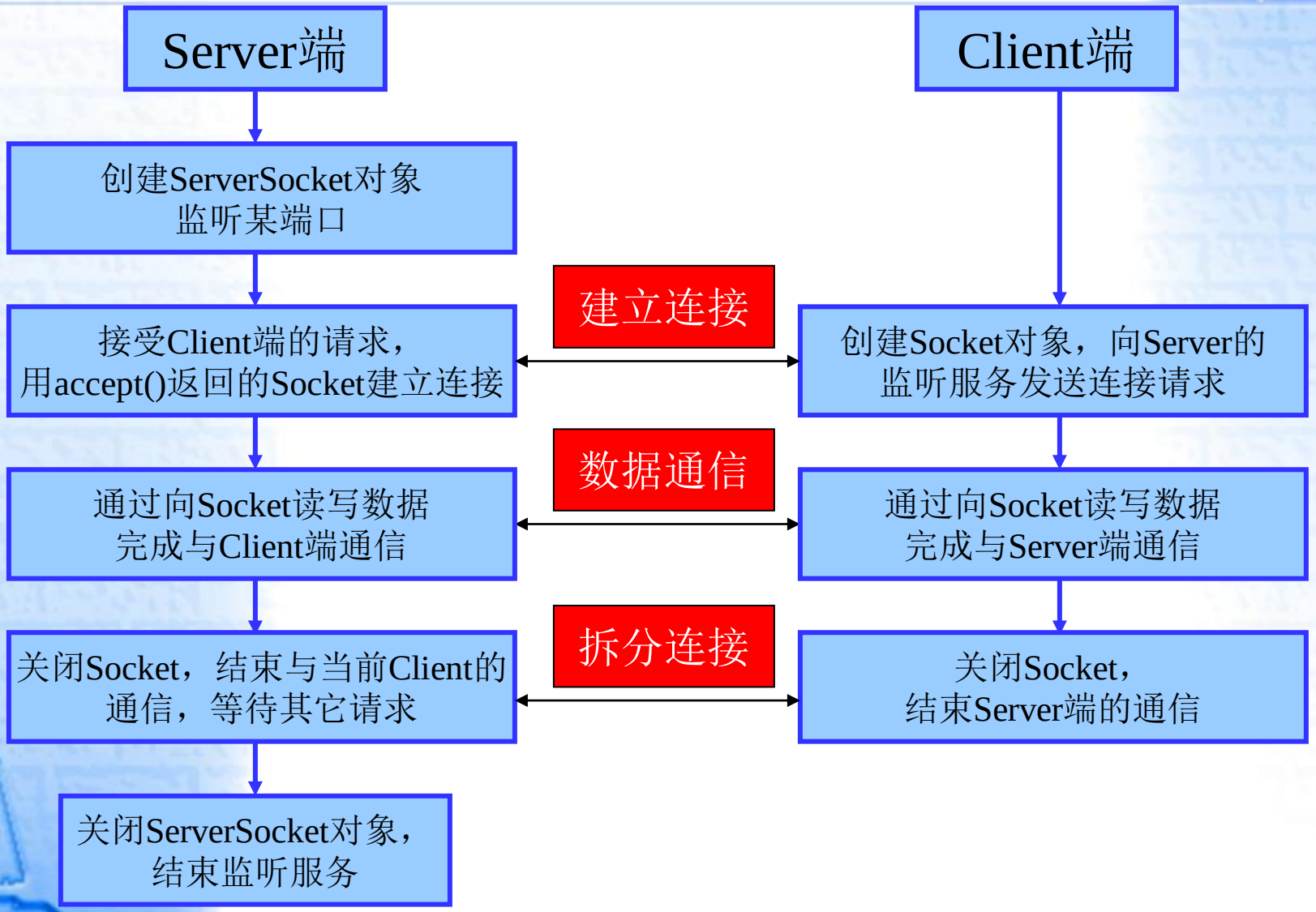
Socket类

- 建立客户机
  - 构造方法
    - Socket(String host, int port)
    - Socket(InetAddress address, int port)
    - ……
    - 参数：主机名、要连接的主机端口号、指定主机的IP地址
  - 常用的方法
    - getInputStream()，创建输入流
    - getOutputStream()，创建输出流
    - getInetAddress()，获得Socket连接的IP地址
    - getLocalAddress()，获得本地机使用的IP地址
    - close()，关闭Socket

基于Socket的网络通信

- 当程序需要建立网络连接时，必须有一台机器运行一个程序，随时等候连接，而另一端的程序则对其发出连接请求，同电话系统类似
- 建立连接的过程
  - 服务器端生成一个**ServerSocket实例对象**，随时监听客户端的连接请求
  - 客户端生成一个**Socket实例对象**，并发出连接请求
  - 服务器端通过**accept()方法**接收到客户端的请求后，开辟一个接口与之进行连接，并生成所需的I/O数据流
  - 通信都是通过一对**InputStream()**和**OutputStream()**进行的。通信结束后，两端分别关闭对应的Socket接口

Socket通信过程

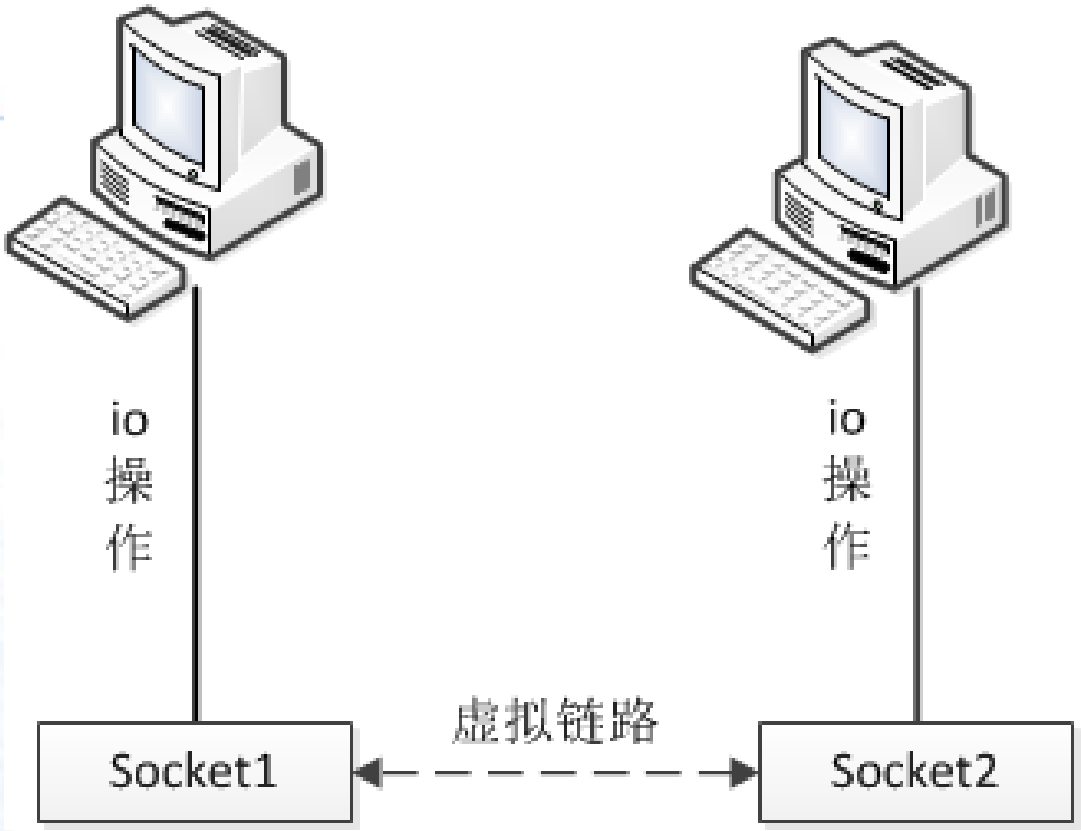

Server端
Client端
创建ServerSocket对象
监听某端口
建立连接
接受Client端的请求，
用accept()返回的Socket建立连接
创建Socket对象，向Server的
监听服务发送连接请求
数据通信
通过向Socket读写数据
完成与Client端通信
通过向Socket读写数据
完成与Server端通信
拆分连接
关闭Socket，结束与当前Client的
通信，等待其它请求
关闭Socket，
结束Server端的通信
关闭ServerSocket对象，
结束监听服务

```
//创建一个ServerSocket，用于监听客户端的连接请求
ServerSocket ss = new ServerSocket(8080);
//采用循环不断接受来自客户端的请求
while(true) {
    //每当接受到客户端Socket的请求，服务器端也对应产生一个Socket
    Socket sServer = ss.accept();
    //下面就可以使用Socket进行通信了
    ……  }
```

服务器端

```
//创建连接到服务器所在ip、所绑定端口号的Socket
Socket sClient = new Socket("127.0.0.1", 8080);
//下面就可以使用Socket进行通信了
    ……
```

客户端



Socket类:

InputStream getInputStream();

OutputStream getOutputStream();

简单的服务器与客户的交互

- Network_5:

EchoServer” & “EchoClient“

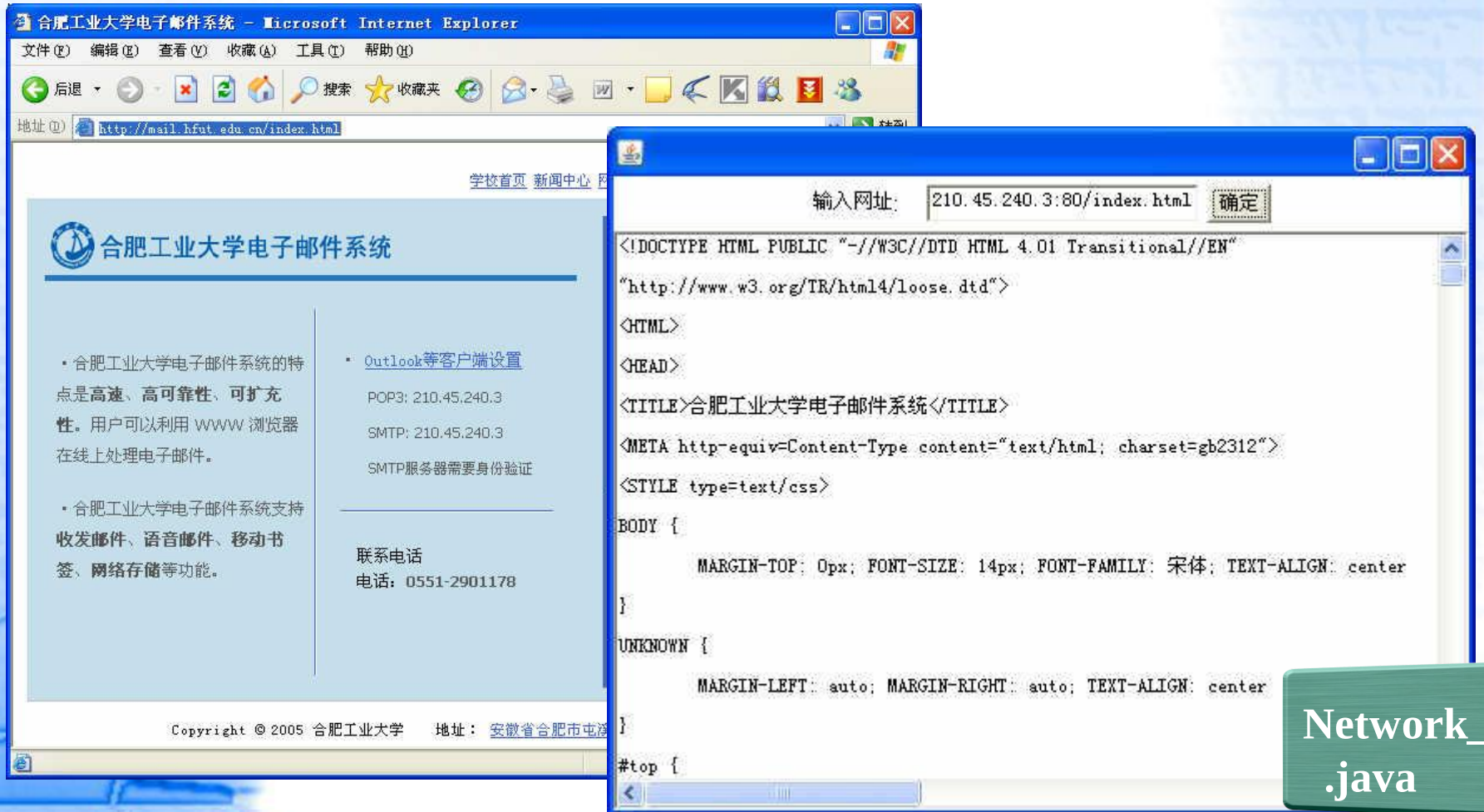
C/S方式，C发送消息，S接收消息并显示

URL与 Socket 通信的区别

- 都是利用建立连接、获取流来进行通信，区别在于
- 通信方式
  - 利用socket进行通信时，在服务器端运行一个socket通信程序。服务器端不停地监听某个端口，等待客户的连接申请，接到申请后建立连接并进行通信，所以，在socket通信方式中，服务器是**主动**等待连接通信的到来。
  - 利用URL进行通信时，在服务器端常驻一个CGI（Common Gateway Interface，公共网关接口）程序，一直处于休眠状态。只有在客户端要求建立连接时才被激活，然后与用户进行通信。所以在URL通信方式中，服务器是**被动**等待连接通信的到来。
- 连接数
  - Socket服务器可以**同时与多个客户端**进行相互通信
  - URL通信方式是服务器**只能与一个客户端**进行通信。

“GUI” 浏览器

- 基本的图形化URL程序 AWT + URL对象
- 访问 http://210.45.240.3:80/index.html



Network_4.java

基于UDP的Socket通信

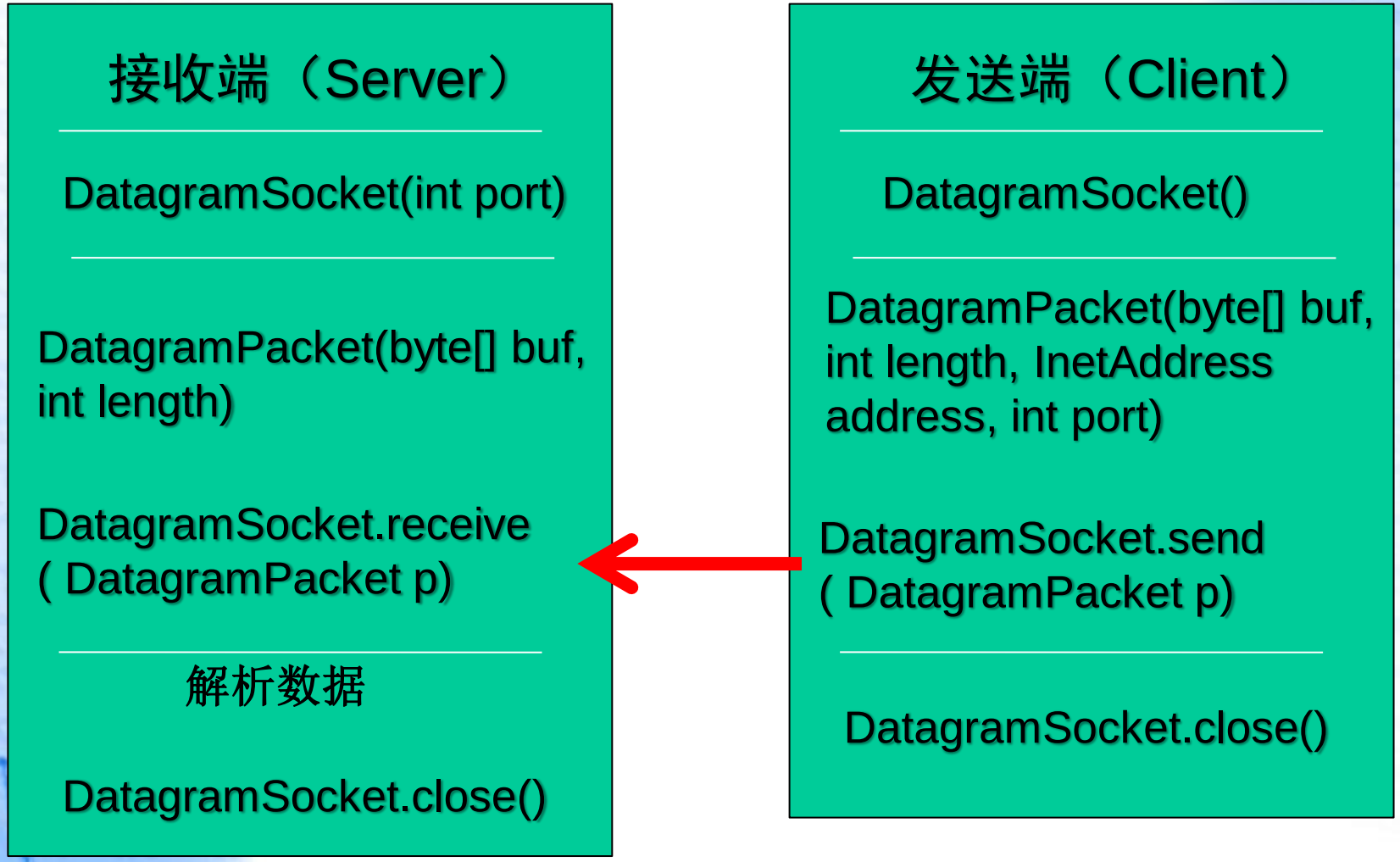
DatagramSocket类

- 此类表示用来发送和接收数据报包的套接字，启用 UDP 广播发送
  - 构造方法
    - **DatagramSocket**()
    - **DatagramSocket**(int port)
    - **DatagramSocket**(int port, InetAddress laddr)
  - 常用的方法
    - send(**DatagramPacket** p)，发送数据包
    - receive(**DatagramPacket** p)，接收数据包
    - getInetAddress()，返回此套接字连接的地址
    - disconnect()，断开套接字的连接
    - close()，关闭此数据报套接字
    - ……

DatagramPacket类

- 构造数据报包
  - 构造方法
    - DatagramPacket(byte[] buf, int length)
      用来接收长度为 length 的数据包。
    - DatagramPacket(byte[] buf, int length, InetAddress address, int port)
      用来将长度为 length 的包发送到指定主机上的指定端口号
  - 常用方法
    - getData()，返回数据包中的数据
    - getAddress()，返回机器的 IP 地址，此数据报将要发往该机器或者是从该机器接收到的
    - getPort()，返回主机的端口号
    - ……

基于UDP的socket编程



基于UDP的socket编程

- 接收端程序编写
  - 调用DatagramSocket(int port)创建一个数据报套接字，并绑定到指定端口上
  - 调用DatagramPacket，建立一个字节数组以接收UDP包
  - 调用DatagramSocket类的receive()，接收UDP包
  - 关闭数据报套接字
- 发送端程序编写
  - 调用DatagramSocket()创建一个数据报套接字
  - 调用DatagramPacket建议要发送的UDP包
  - 调用DatagramScoket类的send()，发送UDP包
  - 关闭数据报套接字

UDPDemo

编程示例

- 基于UDP的方式建立Server端和Client端，进行简单的通信。由Client端向Server端发送数据，Server端接收数据后将其打印输出

Network_6.java

本章总结

- 本章学习Java语言在网络通讯方面的应用
  - 网络基本概念
  - java.net中类的使用
  - 简单的socket程序设计

本章内容总结

- 了解基本概念
  - 网络、IP地址、端口、URL
- 了解数据传输过程
  - 协议：TCP和UDP
- 熟悉相关类的使用
  - InetAddress、InetSocketAddress
  - TCP：Socket、ServerSocket
  - UDP：DatagramSocket，DatagramPacket
  - 通信的底层还是流（IO），只不过对其进行了封装

Class is Dismissed!

